

Lodging Transport Management System

Kiến trúc tổng thể hệ thống của bạn như thế nào?

Kiến trúc tổng thể của hệ thống mình được xây dựng theo mô hình microservices kết hợp event-driven, chạy trên AWS. Ở mức tổng quan, luồng hoạt động chính là: người dùng thao tác trên web (React), request đi qua Nginx (đóng vai trò reverse proxy), sau đó được chuyển đến các backend microservices viết bằng FastAPI. Mỗi service xử lý logic riêng và làm việc với database tương ứng của nó.

Hệ thống được chia thành nhiều service chính. Ví dụ, Booking Service chịu trách nhiệm tạo, cập nhật và hủy booking khách sạn hoặc phương tiện di chuyển, đồng thời validate dữ liệu trước khi lưu vào PostgreSQL. Notification Service chuyên xử lý việc gửi email xác nhận thông qua AWS SES khi có booking mới hoặc thay đổi trạng thái. Reporting Service dùng để tổng hợp dữ liệu và xuất báo cáo cho admin, có thể truy vấn từ PostgreSQL hoặc Elasticsearch để tối ưu tìm kiếm và thống kê. Ngoài ra còn có các service tích hợp để đồng bộ dữ liệu với hệ thống đối tác bên ngoài.

Về cách các service giao tiếp với nhau, mình sử dụng cả hai hình thức synchronous và asynchronous. Với synchronous communication, các service gọi nhau qua REST API khi cần phản hồi ngay lập tức, ví dụ như frontend gọi Booking Service để tạo booking và nhận kết quả ngay. Còn với asynchronous communication, mình sử dụng AWS SQS và SNS để publish và subscribe event. Ví dụ, khi một booking được tạo thành công, Booking Service sẽ publish một event. Notification Service sẽ nhận event đó để gửi email, còn Reporting Service có thể dùng event để cập nhật dữ liệu thống kê. Cách này giúp hệ thống giảm phụ thuộc trực tiếp giữa các service và tăng khả năng chịu tải.

Về database, mình áp dụng nguyên tắc database per service, tức là mỗi service sở hữu database riêng thay vì dùng chung một database. Booking Service dùng PostgreSQL cho dữ liệu giao dịch, một số dữ liệu linh hoạt hoặc log có thể dùng DynamoDB, còn Elasticsearch phục vụ cho tìm kiếm nhanh. Việc tách database giúp giảm coupling, tránh xung đột schema và cho phép scale từng service độc lập. Tuy nhiên, nó dẫn đến mô hình eventual consistency, nên mình xử lý đồng bộ dữ liệu thông qua event-driven.

Lý do mình chọn microservices thay vì monolith là vì hệ thống có nhiều domain khác nhau như booking, transport, notification, reporting và tích hợp bên thứ ba. Nếu xây dựng theo monolith, khi một phần gặp vấn đề có thể ảnh hưởng toàn hệ thống, và việc scale cũng phải scale toàn bộ ứng dụng. Với microservices, mỗi phần có thể phát triển, deploy và scale độc lập, phù hợp với môi trường cloud như AWS và định hướng mở rộng lâu dài.

Bạn đã implement event-driven pattern như thế nào?

Trong dự án này, mình triển khai event-driven pattern để các service có thể giao tiếp với nhau một cách linh hoạt và giảm phụ thuộc trực tiếp. Thay vì service này gọi thẳng service kia và chờ phản hồi, mình cho các service “nói chuyện” với nhau thông qua event. Khi một hành động quan trọng xảy ra, service sẽ phát ra một event, còn service nào quan tâm thì sẽ lắng nghe và xử lý.

Cụ thể, mình sử dụng Amazon SQS làm message queue và Amazon SNS để publish-subscribe event. Khi có một booking mới được tạo thành công, Booking Service sẽ publish một event như “BookingCreated” lên SNS. SNS sau đó sẽ gửi event này đến các SQS queue đã subscribe trước đó. Ví dụ, Notification Service sẽ subscribe vào queue này để nhận event và thực hiện gửi email xác nhận thông qua AWS SES. Nếu sau này cần thêm service khác như Analytics hoặc Audit Log, mình chỉ cần cho service đó subscribe vào SNS mà không cần sửa Booking Service.

Về cấu trúc event (event payload), mình thiết kế theo dạng JSON, bao gồm các thông tin cần thiết như booking_id, user_id, booking_status, timestamp và một số metadata. Mình cố gắng giữ payload đủ thông tin để service nhận event không cần phải gọi ngược lại Booking Service quá nhiều lần. Ngoài ra mình có version field trong event để hỗ trợ việc nâng cấp schema sau này.

Để tránh xử lý trùng lặp event, mình áp dụng nguyên tắc idempotency. Ví dụ, Notification Service sẽ lưu lại event_id hoặc booking_id đã xử lý vào database hoặc cache. Nếu nhận lại cùng một event (do retry hoặc message bị gửi lại), service sẽ kiểm tra và bỏ qua nếu đã xử lý trước đó. Điều này giúp đảm bảo hệ thống không gửi email trùng hoặc cập nhật dữ liệu sai.

Về retry và lỗi, SQS hỗ trợ cơ chế retry tự động nếu message chưa được xử lý thành công. Nếu service xử lý lỗi (ví dụ lỗi tạm thời khi gửi email), message sẽ quay lại queue sau một khoảng thời gian. Sau một số lần retry nhất định mà vẫn thất bại, message sẽ được chuyển sang Dead Letter Queue (DLQ). Mình có monitor DLQ để kiểm tra và xử lý thủ công hoặc fix bug nếu cần. Cách này giúp hệ thống không bị mất message và vẫn đảm bảo độ tin cậy cao.

Tóm lại, event-driven pattern trong hệ thống giúp các service tách biệt rõ ràng, dễ mở rộng, tăng khả năng chịu lỗi và phù hợp với kiến trúc microservices trên AWS.

Làm sao đảm bảo data consistency giữa các service?

Trong hệ thống microservices, vì mỗi service có database riêng nên mình không thể dùng transaction chung như trong monolith. Thay vào đó, mình áp dụng mô hình eventual

consistency, nghĩa là dữ liệu giữa các service có thể không đồng bộ ngay lập tức, nhưng sau một khoảng thời gian ngắn thì sẽ trở nên nhất quán thông qua cơ chế event-driven.

Cụ thể, khi một thay đổi quan trọng xảy ra, ví dụ tạo booking thành công, Booking Service sẽ cập nhật database của nó trước, sau đó publish một event như “BookingCreated”. Các service khác như Notification hoặc Reporting sẽ nhận event này và cập nhật dữ liệu của mình tương ứng. Nhờ vậy, dữ liệu được lan truyền dần sang các service khác mà không cần transaction đồng bộ giữa nhiều database.

Trong những flow phức tạp hơn, ví dụ như cần thực hiện nhiều bước liên tiếp (tạo booking, giữ phòng, xác nhận thanh toán...), mình có thể áp dụng tư duy của Saga pattern. Nghĩa là mỗi bước sẽ publish event cho bước tiếp theo. Nếu một bước thất bại, hệ thống sẽ thực hiện các hành động bù (compensation action), ví dụ hủy booking nếu thanh toán không thành công. Điều này giúp đảm bảo tính nhất quán về mặt nghiệp vụ mà không cần distributed transaction.

Để tăng độ tin cậy, mình sử dụng cơ chế retry và Dead Letter Queue (DLQ). Nếu một service xử lý event bị lỗi tạm thời, message sẽ được retry tự động. Nếu sau nhiều lần retry vẫn thất bại, message sẽ được chuyển sang DLQ để kiểm tra và xử lý sau. Nhờ vậy, hệ thống không bị mất dữ liệu hoặc bỏ sót event.

Ngoài ra, mình áp dụng nguyên tắc idempotent consumer, tức là mỗi service khi nhận event sẽ kiểm tra xem event đó đã được xử lý trước đó chưa. Ví dụ, nếu cùng một event bị gửi lại do retry, service sẽ dựa vào event_id hoặc booking_id để tránh xử lý trùng. Điều này rất quan trọng để tránh gửi email hai lần hoặc ghi dữ liệu sai.

Cuối cùng, mình có versioning cho event schema. Trong payload của event luôn có một trường version để khi cần thay đổi cấu trúc dữ liệu, các service cũ vẫn có thể xử lý được hoặc có thời gian nâng cấp dần. Điều này giúp hệ thống ổn định và tránh lỗi khi mở rộng trong tương lai.

Tóm lại, mình đảm bảo data consistency bằng cách kết hợp eventual consistency, event-driven, retry và DLQ, idempotent processing và quản lý version của event. Cách này phù hợp với kiến trúc microservices và giúp hệ thống vừa linh hoạt vừa đáng tin cậy.

Khi tối ưu SQL queries, bạn đã làm gì cụ thể?

Khi tối ưu SQL queries trong dự án, mình tập trung vào việc tìm ra nguyên nhân khiến query chậm trước, sau đó mới cải thiện từng phần. Việc đầu tiên mình làm là sử dụng EXPLAIN query plan của PostgreSQL để xem database đang thực thi câu lệnh như thế

nào. Nhờ đó mình biết được query có đang scan toàn bộ bảng (full table scan) hay có sử dụng index hay không. Nếu thấy query tốn nhiều thời gian do scan quá nhiều dữ liệu, mình sẽ tìm cách tối ưu lại cấu trúc hoặc thêm index phù hợp.

Một cải thiện quan trọng là thêm index, đặc biệt là composite index cho những cột thường xuyên được dùng chung trong điều kiện WHERE hoặc ORDER BY, ví dụ như `booking_status` và `created_at`. Việc thêm index giúp database tìm dữ liệu nhanh hơn thay vì phải duyệt toàn bộ bảng. Tuy nhiên mình cũng cân nhắc kỹ vì thêm quá nhiều index có thể làm chậm thao tác ghi (INSERT/UPDATE).

Trong quá trình làm việc với SQLAlchemy, mình cũng gặp vấn đề N+1 query. Ví dụ, khi lấy danh sách booking và mỗi booking lại load thêm thông tin user hoặc hotel, nếu không cấu hình đúng sẽ dẫn đến việc mỗi record lại tạo thêm một query riêng. Để tránh điều này, mình sử dụng eager loading (joinedload hoặc selectinload trong SQLAlchemy) để load dữ liệu liên quan ngay từ đầu trong một số query ít nhất có thể. Điều này giúp giảm số lượng query gửi xuống database đáng kể.

Ngoài ra, mình không bao giờ load toàn bộ dữ liệu khi không cần thiết. Với những API trả về danh sách booking hoặc báo cáo, mình luôn áp dụng pagination (limit và offset hoặc cursor-based pagination). Điều này giúp giảm tải cho database và giảm dung lượng dữ liệu trả về cho frontend, cải thiện tốc độ phản hồi API.

Cuối cùng, với những dữ liệu cần tìm kiếm nhanh hoặc được truy vấn nhiều lần, mình tận dụng caching hoặc Elasticsearch. Ví dụ, những chức năng tìm kiếm booking theo nhiều điều kiện phức tạp sẽ được index sang Elasticsearch để truy vấn nhanh hơn. Với một số dữ liệu ít thay đổi nhưng được đọc thường xuyên, có thể cache tạm thời để giảm tải cho database chính.

Tóm lại, mình tối ưu SQL bằng cách phân tích execution plan, thêm index hợp lý, tránh N+1 query, áp dụng pagination và sử dụng caching hoặc search engine khi cần. Mục tiêu là giảm thời gian phản hồi API và đảm bảo hệ thống vẫn ổn định khi lượng dữ liệu tăng lên.

Vai trò của DynamoDB và PostgreSQL trong hệ thống là gì?

Trong hệ thống của mình, PostgreSQL và DynamoDB được sử dụng cho hai mục đích khác nhau, dựa trên đặc tính của từng loại dữ liệu.

PostgreSQL được dùng cho transactional data, tức là những dữ liệu quan trọng và cần tính chính xác cao như booking khách sạn, trạng thái booking, thông tin thanh toán, lịch sử thay đổi trạng thái. Đây là những dữ liệu có quan hệ rõ ràng giữa các bảng và cần đảm bảo

tính toàn vẹn (ACID). Ví dụ, khi tạo một booking, mình cần đảm bảo dữ liệu được ghi chính xác, không bị trùng, và có thể rollback nếu có lỗi. PostgreSQL rất phù hợp cho các nghiệp vụ dạng này vì nó hỗ trợ transaction mạnh và truy vấn quan hệ phức tạp tốt.

Còn DynamoDB mình sử dụng cho những dữ liệu có tần suất ghi/đọc cao hoặc cấu trúc linh hoạt, ví dụ như log sự kiện, metadata, hoặc một số dữ liệu trung gian phục vụ đồng bộ. Những loại dữ liệu này thường không cần join phức tạp và có thể thay đổi cấu trúc theo thời gian. DynamoDB là NoSQL nên không cần schema cố định như PostgreSQL, giúp mình linh hoạt hơn khi lưu trữ. Ngoài ra, DynamoDB có khả năng scale rất tốt và xử lý được lưu lượng lớn mà không cần quản lý nhiều về hạ tầng.

Lý do mình chọn NoSQL như DynamoDB là vì tính scalability và flexible schema. Khi dữ liệu tăng nhanh hoặc cần xử lý nhiều request song song, DynamoDB có thể mở rộng dễ dàng theo chiều ngang. Đồng thời, nếu cần thêm field mới vào dữ liệu log hoặc metadata, mình không cần migrate schema phức tạp như trong relational database.

Tuy nhiên, giữa hai loại database này cũng có trade-off về consistency. PostgreSQL cung cấp strong consistency và transaction đầy đủ, rất phù hợp cho dữ liệu cốt lõi như booking. Trong khi đó, DynamoDB có thể cấu hình strong consistency nhưng thường được sử dụng theo hướng eventual consistency để tối ưu hiệu năng. Vì vậy, mình chỉ dùng DynamoDB cho những dữ liệu không yêu cầu transaction phức tạp hoặc không ảnh hưởng trực tiếp đến nghiệp vụ chính.

Tóm lại, PostgreSQL xử lý dữ liệu nghiệp vụ chính cần độ chính xác cao, còn DynamoDB hỗ trợ các dữ liệu linh hoạt, throughput lớn và dễ mở rộng. Việc kết hợp cả hai giúp hệ thống vừa đảm bảo tính ổn định, vừa tối ưu hiệu năng và khả năng scale.

Elasticsearch được dùng trong trường hợp nào?

Trong hệ thống của mình, Elasticsearch được sử dụng chủ yếu cho các chức năng tìm kiếm nhanh và linh hoạt, đặc biệt là khi cần full-text search hoặc lọc theo nhiều điều kiện cùng lúc. Ví dụ, khi người dùng hoặc admin muốn tìm booking theo tên khách hàng, email, mã booking, trạng thái, khoảng thời gian... thì nếu chỉ dùng PostgreSQL với nhiều điều kiện phức tạp, hiệu năng có thể giảm khi dữ liệu lớn. Elasticsearch được tối ưu cho search nên phản hồi nhanh hơn trong các trường hợp này.

Một trường hợp điển hình là full-text search. Ví dụ, tìm booking theo tên khách sạn hoặc thông tin khách hàng có thể nhập không chính xác 100% (gần đúng). Elasticsearch hỗ trợ phân tích text (text analyzer), cho phép tìm kiếm gần đúng, không phân biệt hoa thường,

hoặc theo từ khóa tương tự. Điều này PostgreSQL có thể làm được nhưng không tối ưu bằng khi dữ liệu lớn.

Ngoài ra, Elasticsearch rất mạnh khi cần lọc theo nhiều filter kết hợp, ví dụ: `booking_status = "CONFIRMED"`, `created_at` trong 30 ngày gần nhất, thuộc một chi nhánh cụ thể, và sắp xếp theo thời gian. Những truy vấn kiểu này được Elasticsearch xử lý rất nhanh nhờ cơ chế inverted index.

Về cách đồng bộ dữ liệu, mình không ghi trực tiếp vào Elasticsearch từ frontend. Thay vào đó, dữ liệu booking vẫn được lưu chính thức trong PostgreSQL trước. Sau đó, mình đồng bộ sang Elasticsearch thông qua event-driven hoặc cron job. Ví dụ, khi có event “BookingCreated” hoặc “BookingUpdated”, một service sẽ nhận event và cập nhật lại document tương ứng trong Elasticsearch. Cách này đảm bảo PostgreSQL vẫn là nguồn dữ liệu chính (source of truth), còn Elasticsearch phục vụ cho mục đích tìm kiếm.

Khi thiết kế index trong Elasticsearch, mình định nghĩa mapping rõ ràng cho từng field, ví dụ field nào là keyword (để filter chính xác), field nào là text (để full-text search), và field ngày tháng để phục vụ sort hoặc range query. Việc thiết kế mapping đúng ngay từ đầu rất quan trọng vì nó ảnh hưởng trực tiếp đến hiệu năng tìm kiếm và độ chính xác của kết quả.

Tóm lại, mình dùng Elasticsearch khi cần tìm kiếm nhanh, full-text search và filter phức tạp trên lượng dữ liệu lớn. PostgreSQL vẫn là database chính cho transaction, còn Elasticsearch đóng vai trò là search engine để tối ưu trải nghiệm người dùng và hiệu năng hệ thống.

Bạn xử lý scheduled tasks và cron jobs như thế nào?

Trong hệ thống của mình, scheduled tasks và cron jobs được dùng cho những công việc cần chạy định kỳ thay vì chạy theo request của người dùng. Ví dụ như tự động hủy booking nếu quá thời gian thanh toán, gửi nhắc nhở trước ngày check-in, hoặc đồng bộ dữ liệu với hệ thống đối tác khách sạn.

Cách mình triển khai phổ biến nhất là sử dụng AWS Lambda kết hợp với EventBridge (CloudWatch Events). Mình tạo một rule theo lịch, ví dụ mỗi 5 phút hoặc mỗi 1 giờ. Khi đến thời điểm đó, EventBridge sẽ tự động trigger một Lambda function. Lambda sẽ chạy logic cần thiết, ví dụ tìm các booking đang ở trạng thái “PENDING” nhưng đã quá hạn thanh toán, sau đó cập nhật trạng thái thành “CANCELLED” và có thể publish event để các service khác xử lý tiếp. Cách này có ưu điểm là không cần quản lý server và dễ scale.

Trong một số trường hợp khác, nếu logic đơn giản và gắn chặt với service, mình có thể dùng cron job chạy trực tiếp trong container (ví dụ chạy background scheduler trong ứng dụng). Tuy nhiên, mình ưu tiên Lambda vì nó tách biệt khỏi service chính và tránh ảnh hưởng hiệu năng API.

Để tránh việc job chạy trùng, mình thiết kế theo hướng idempotent. Nghĩa là trước khi cập nhật dữ liệu, luôn kiểm tra điều kiện rõ ràng, ví dụ chỉ hủy booking nếu trạng thái vẫn là “PENDING”. Nếu job vô tình chạy lại, nó sẽ không ảnh hưởng đến những booking đã xử lý rồi. Ngoài ra, có thể sử dụng database lock hoặc một cờ đánh dấu để đảm bảo một job không bị nhiều instance xử lý cùng lúc.

Tóm lại, mình dùng Lambda + EventBridge cho các tác vụ định kỳ như auto-cancel booking hoặc sync dữ liệu. Mình luôn thiết kế logic an toàn và idempotent để đảm bảo hệ thống ổn định ngay cả khi job bị retry hoặc trigger lại.

Quy trình CI/CD của bạn hoạt động ra sao?

Quy trình CI/CD trong dự án của mình được xây dựng dựa trên GitLab CI pipeline, mục tiêu là tự động hóa toàn bộ quá trình từ lúc developer push code cho đến khi deploy lên môi trường test hoặc SIT. Điều này giúp giảm lỗi thủ công và đảm bảo mỗi lần release đều nhất quán.

Khi developer push code lên repository (ví dụ tạo merge request hoặc merge vào branch chính), GitLab CI sẽ tự động kích hoạt pipeline. Bước đầu tiên trong pipeline thường là build và kiểm tra code. Hệ thống sẽ cài đặt dependency và chạy unit test để đảm bảo các chức năng chính vẫn hoạt động đúng. Nếu test fail thì pipeline sẽ dừng lại và không cho deploy tiếp.

Sau khi test pass, pipeline sẽ tiến hành build Docker image cho service. Mỗi microservice có một Dockerfile riêng, và image sẽ được build theo version cụ thể (ví dụ theo commit hash hoặc tag). Sau đó image được push lên container registry để lưu trữ.

Tiếp theo là bước deploy. Tùy môi trường, pipeline sẽ deploy Docker image mới lên EC2 hoặc container đang chạy. Ví dụ, pipeline cho môi trường Test sẽ tự động pull image mới và restart service. Với môi trường SIT (System Integration Testing), quy trình có thể yêu cầu manual approval trước khi deploy để đảm bảo kiểm soát chất lượng.

Hệ thống của mình có tách rõ các môi trường: Dev, Test và SIT. Mỗi môi trường có cấu hình riêng như database, biến môi trường và endpoint khác nhau. Việc tách môi trường

giúp team test kỹ trước khi đưa lên production, đồng thời tránh ảnh hưởng lẫn nhau giữa các giai đoạn phát triển.

Tóm lại, quy trình CI/CD của mình gồm các bước chính: push code → chạy test → build Docker image → push image → deploy lên môi trường tương ứng. Nhờ vậy, việc release trở nên nhanh, ổn định và giảm rủi ro so với cách deploy thủ công.

Bạn đã xử lý logging và monitoring như thế nào?

Trong hệ thống microservices, logging và monitoring rất quan trọng vì khi có lỗi xảy ra, mình không thể chỉ nhìn vào một service duy nhất mà phải theo dõi toàn bộ luồng xử lý. Vì vậy, mình triển khai logging và monitoring ngay từ đầu để dễ quan sát và debug khi cần.

Về monitoring hiệu năng, mình sử dụng New Relic để theo dõi tình trạng hệ thống. New Relic giúp mình xem được CPU, memory, throughput, số lượng request, và đặc biệt là thời gian phản hồi (API latency) của từng service. Nếu một API nào đó phản hồi chậm bất thường, mình có thể vào dashboard để xem chi tiết transaction đó đang tốn thời gian ở bước nào, ví dụ ở database query hay khi gọi sang service khác.

Về logging, mình áp dụng structured logging thay vì log dạng text tự do. Nghĩa là log được ghi theo dạng JSON với các field rõ ràng như timestamp, service_name, log_level, message, request_id. Cách này giúp dễ filter và tìm kiếm log hơn khi có sự cố. Ví dụ, mình có thể lọc tất cả log level “ERROR” trong khoảng thời gian cụ thể để tìm nguyên nhân lỗi.

Mình cũng theo dõi API latency và error rate. Nếu tỷ lệ lỗi (ví dụ HTTP 5xx) tăng vượt một ngưỡng nhất định, hệ thống sẽ gửi alert để team kiểm tra ngay. Điều này giúp phát hiện vấn đề sớm thay vì đợi người dùng báo lỗi.

Một điểm quan trọng trong microservices là sử dụng Correlation ID. Khi một request từ frontend đi qua nhiều service khác nhau, mình sẽ tạo một correlation_id ngay từ đầu và truyền nó qua header của các request nội bộ. Nhờ vậy, khi cần debug, mình có thể tìm tất cả log liên quan đến cùng một request chỉ bằng cách search theo correlation_id. Điều này giúp trace toàn bộ luồng xử lý từ Booking Service sang Notification hoặc Reporting Service rất dễ dàng.

Tóm lại, mình kết hợp New Relic để monitor hiệu năng và health của hệ thống, structured logging để dễ truy vết, alert để phát hiện lỗi sớm, và correlation ID để trace request xuyên suốt nhiều service. Nhờ vậy, việc vận hành và xử lý sự cố trở nên nhanh và hiệu quả hơn.

Một challenge kỹ thuật lớn bạn gặp trong dự án là gì?

Một challenge kỹ thuật lớn mình gặp trong dự án là vấn đề race condition khi tạo booking. Cụ thể, trong một số trường hợp hiếm, nếu hai request gần như được gửi cùng lúc cho cùng một tài nguyên (ví dụ cùng một phòng hoặc cùng một slot thời gian), hệ thống có thể xử lý song song và dẫn đến việc tạo booking trùng hoặc trạng thái không chính xác. Điều này xảy ra vì microservices xử lý bất đồng bộ và có nhiều instance chạy song song.

Để debug vấn đề này, mình kiểm tra log theo correlation ID để xem luồng xử lý của từng request. Sau đó mình phân tích lại logic database và nhận ra cần kiểm soát chặt hơn ở tầng transaction. Giải pháp mình áp dụng là sử dụng database transaction kết hợp với cơ chế lock (ví dụ select for update) hoặc kiểm tra điều kiện ràng buộc ngay tại database level. Nhờ vậy, nếu hai request cùng lúc cố gắng cập nhật một tài nguyên, database sẽ đảm bảo chỉ một request thành công.

Một vấn đề khác mình từng gặp là duplicate event trong hệ thống event-driven. Do cơ chế retry của SQS, đôi khi cùng một event có thể được gửi lại nếu lần xử lý trước đó chưa xác nhận thành công. Điều này dẫn đến việc service downstream xử lý hai lần, ví dụ gửi email xác nhận trùng. Để xử lý, mình áp dụng nguyên tắc idempotent consumer, tức là mỗi event đều có event_id riêng, và service sẽ kiểm tra xem event đó đã được xử lý chưa trước khi thực hiện logic.

Ngoài ra, mình cũng gặp tình huống third-party API phản hồi chậm hoặc không ổn định. Điều này làm ảnh hưởng đến thời gian phản hồi của hệ thống. Giải pháp mình áp dụng là thêm timeout rõ ràng khi gọi API, kết hợp retry có kiểm soát, và trong một số trường hợp dùng cơ chế fallback để tránh block toàn bộ flow chính.

Tóm lại, challenge lớn nhất đến từ việc hệ thống chạy phân tán và bất đồng bộ. Mình giải quyết bằng cách phân tích log kỹ lưỡng, kiểm soát transaction ở database, đảm bảo idempotency trong event processing và thêm cơ chế retry, timeout hợp lý khi tích hợp bên ngoài. Qua đó hệ thống trở nên ổn định và đáng tin cậy hơn.

TỔNG HỢP CÂU HỎI:

Kiến trúc tổng thể hệ thống của bạn như thế nào?

Bạn đã implement event-driven pattern như thế nào?

Làm sao đảm bảo data consistency giữa các service?

Khi tối ưu SQL queries, bạn đã làm gì cụ thể?

Vai trò của DynamoDB và PostgreSQL trong hệ thống là gì?

Elasticsearch được dùng trong trường hợp nào?

Bạn xử lý scheduled tasks và cron jobs như thế nào?

Quy trình CI/CD của bạn hoạt động ra sao?

Bạn đã xử lý logging và monitoring như thế nào?

Một challenge kỹ thuật lớn bạn gặp trong dự án là gì?

Bây giờ tôi sẽ đưa bạn từng câu hỏi, bạn hãy giúp tôi trả lời bằng 1 cách dễ hiểu nhất vì vốn tôi cũng không có quá rành. Viết câu trả lời bằng các đoạn văn bản nhé, nhớ là đoạn.

Một challenge kỹ thuật lớn bạn gặp trong dự án là gì?

Hint trả lời (ví dụ):

Race condition khi booking.

Duplicate event.

Third-party API chậm.

Data mismatch giữa services.

Cách bạn debug và giải quyết.